



## **UNIVERSIDAD MAYOR DE SAN SIMÓN**

Facultad de Ciencias y Tecnología

Carrera de Ingeniería de Sistemas / Informática



### **PROYECTO FINAL DE PROGRAMACIÓN MÓVIL**

# **Caja Cordes Bolivia – Sistema de Circulares**

**Guía de configuración, compilación y generación del APK**

**Docente:**

Dr. Américo Fiorilo Lozada Ph.D.

**Integrante del grupo:**

Wálter Guzmán Méndez

Fecha de presentación: 5 de junio de 2026

Cochabamba – Bolivia

Aplicación **Circulares** — difusión de circulares e instructivos.

Kotlin + MVVM + Firebase (Firestore y Storage) + WorkManager.

"Se Desarrolla una aplicación nativa para Android utilizando Android Studio con lenguaje Kotlin y arquitectura MVVM, que cumpla con las siguientes funcionalidades específicas:

Funcionalidad principal: Difusión de circulares e instructivos.

Acceso de usuarios: Solo el administrador requiere inicio de sesión (usuario y contraseña definibles en el código o mediante pantalla de configuración). El resto de usuarios solo instala la aplicación y no necesita registrarse ni iniciar sesión.

Carga de archivos: El administrador, una vez autenticado, debe poder subir archivos en formato imagen (JPG, PNG) o documento PDF que contenga la circular o instructivo.

Visualización automática: Cuando se sube un archivo nuevo, este debe mostrarse automáticamente en la pantalla de todos los dispositivos que tengan la aplicación instalada. La visualización debe repetirse cada 4 horas durante un periodo de 2 días consecutivos desde la fecha de publicación.

Reportes y estadísticas: Sección exclusiva para el administrador donde pueda visualizar cantidad de circulares enviadas, fecha de envío, cantidad de visualizaciones registradas y estado de difusión.

Almacenamiento y conexión: Utilizar Firebase Firestore para almacenar la información de los documentos y Firebase Storage para guardar los archivos. Implementar notificaciones y programación de tareas mediante WorkManager para gestionar la repetición de visualización cada 4 horas por 2 días.

Interfaz: Diseño limpio, moderno y funcional, compatible desde Android 8.0 hasta versiones más recientes.

## 1. Requisitos previos

| Herramienta    | Versión recomendada                |
|----------------|------------------------------------|
| Android Studio | Ladybug (2024.2) o más reciente    |
| JDK            | 17 (incluido en Android Studio)    |
| Android SDK    | API 34 instalada + API 26 mínima   |
| Cuenta Google  | Para Firebase (gratis, plan Spark) |

Min SDK: **26 (Android 8.0)**. Target SDK: **34**.

## 2. Crear el proyecto en Firebase

1. Entra a <https://console.firebase.google.com> y pulsa **Agregar proyecto**.
2. Nombre sugerido: `circulares` (acepta los términos, Analytics es opcional).
3. Dentro del proyecto, pulsa el icono de **Android** para registrar la app:

- **Nombre del paquete:** `com.circulares.difusion` (debe coincidir EXACTAMENTE con el `applicationId` del proyecto).
  - Apodo y SHA-1: opcionales para esta app (no usamos login de Google).
4. Pulsa **Registrar app** y **Descarga** `google-services.json`.
  5. Copia ese archivo dentro de la carpeta `app/` del proyecto, **reemplazando** el `google-services.json` de ejemplo que viene incluido.

Sin el `google-services.json` real, la app compila pero no conecta con Firebase.

## 2.1 Habilitar Firestore

1. En la consola, menú lateral → **Compilación** → **Firestore Database** → **Crear base de datos**.
2. Elige modo **producción** y una ubicación cercana (ej. `us-central` o `southamerica-east1`).
3. Ve a la pestaña **Reglas** y pega estas reglas (lectura pública, escritura solo autenticada-por-app; ajústalas a tu política de seguridad):

```
rules\
_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /circulares/{doc} {
      // Todos pueden leer las circulares (los usuarios no inician sesión)
      allow read: if true;
      // La escritura la realiza el administrador desde la app.
      // Para una app interna controlada, puedes permitir escritura:
      allow write: if true;
    }
  }
}
```

\*Nota de seguridad: `allow write: if true` permite escribir a cualquiera con la app. Es aceptable para una app interna/piloto. Para producción real, integra Firebase Authentication y restringe la escritura a un usuario administrador. La guía de mejoras está al final.

## 2.2 Habilitar Storage

1. Menú lateral → **Compilación** → **Storage** → **Comenzar**.
2. Acepta las reglas iniciales y elige la misma ubicación.
3. En **Reglas**, usa algo equivalente:

```
rules\
_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /circulares/{archivo} {
      allow read: if true;
      allow write: if true;
    }
  }
}
```

}

### 2.3 (Opcional) Cloud Messaging

No es necesario para el funcionamiento básico. La difusión automática ya funciona con el listener en tiempo real de Firestore (app abierta) y con WorkManager (segundo plano, cada 4 h). Si quieres push instantáneo con la app cerrada, habilita **Cloud Messaging** y revisa la sección 8.

### 3. Abrir e importar el proyecto en Android Studio

1. Descomprime `CircularesApp.zip`.
  2. Android Studio → **File** → **Open** → selecciona la carpeta `CircularesApp`.
  3. Espera a que Gradle sincronice (**Gradle Sync**). La primera vez descargará dependencias; necesita internet.
  4. Si aparece "Gradle wrapper", acéptalo. El proyecto ya incluye `gradle-wrapper.jar`, por lo que la sincronización debería ser directa.
- > Si la sincronización falla por el wrapper, ejecuta en una terminal dentro del proyecto: ``gradle wrapper --gradle-version 8.7`` (requiere Gradle instalado) y vuelve a abrir.

### 4. Credenciales del administrador

Por defecto:

- **Usuario:** `admin`
- **Contraseña:** `admin123`

Para cambiarlas tienes dos opciones:

1. **En el código:** edita `app/src/main/java/com/circulares/difusion/util/Config.kt`, campos `ADMIN\_USER` y `ADMIN\_PASSWORD`, y recompila.
2. **En la app:** inicia sesión, abre **Configuración** en el panel del administrador y guarda nuevas credenciales (se almacenan localmente con DataStore y tienen prioridad sobre las del código).

### 5. Ejecutar la app en un dispositivo o emulador

1. Conecta un dispositivo con **depuración USB** activada, o crea un emulador (**Device Manager**) con API 26+.
2. Pulsa el botón **Run**  (Shift+F10).
3. La app abre en la pantalla principal. Sin circulares aún, verás el mensaje de "no hay circulares vigentes".
4. Pulsa el botón **Administrador** → inicia sesión → **Subir circular** → elige una imagen o PDF y publica.
5. La circular aparecerá automáticamente en la pantalla principal de todos los dispositivos con la app instalada (mientras tengan internet).

### 6. Generar el APK de depuración (rápido, para pruebas)

**Desde Android Studio:**

- Menú **Build** → **Build Bundle(s) / APK(s)** → **Build APK(s)**.
- Al terminar, aparece una notificación con enlace **locate**; el archivo está en: ``app/build/outputs/apk/debug/app-debug.apk``

**Desde terminal:**

```
```bash
# Linux/macOS
./gradlew assembleDebug
# Windows
gradlew.bat assembleDebug
```

Este APK es instalable directamente para pruebas internas.

## 7. Generar el APK de release firmado (para distribuir)

Un APK de release debe ir **firmado**.

### 7.1 Crear un keystore (una sola vez)

```
```bash
keytool -genkey -v -keystore circulares.jks -keyalg RSA -keysize 2048 -validity 10000 -
alias circulares
```

Responde a las preguntas y **guarda la contraseña**. Guarda `circulares.jks` fuera del control de versiones.

### 7.2 Firmar desde Android Studio (recomendado)

1. Menú **Build** → **Generate Signed Bundle / APK**.
2. Elige **APK** → **Next**.
3. **Key store path**: selecciona `circulares.jks`, escribe contraseñas y alias.
4. **Next** → marca **release** → marca las casillas V1 y V2 → **Finish**.
5. El APK firmado queda en `app/release/app-release.apk`.

### 7.3 Firmar por línea de comandos (alternativa)

Añade en `app/build.gradle.kts` dentro de `android { }`:

```
```kotlin
signingConfigs {
    create("release") {
        storeFile = file("../circulares.jks")
        storePassword = "TU_PASSWORD"
        keyAlias = "circulares"
        keyPassword = "TU_PASSWORD"
    }
}
buildTypes {
    getByName("release") {
        signingConfig = signingConfigs.getByName("release")
    }
}
```

Luego:

```
```bash
./gradlew assembleRelease
```
```

APK en `app/build/outputs/apk/release/app-release.apk`.

## 8. (Opcional) Push instantáneo con FCM

La app ya difunde automáticamente sin esto. Si quieres una notificación push en el instante en que se publica una circular, incluso con la app cerrada:

1. En la `MainActivity` o `CircularesApplication`, suscribe cada dispositivo al topic:

```
```kotlin
    com.google.firebase.messaging.FirebaseMessaging.getInstance()
        .subscribeToTopic("circulares")
```

2. Crea una **Cloud Function** (Blaze plan) que escuche nuevos documentos en `circulares/` y envíe un mensaje al topic `circulares`. Esqueleto:

```
```javascript
exports.notificarCircular = functions.firestore
    .document('circulares/{id}')
    .onCreate(async (snap) => {
        const data = snap.data();
        await admin.messaging().send({
            topic: 'circulares',
            notification: { title: 'Nueva circular', body: data.titulo }
        });
    });
```

3. El `CircularMessagingService` ya incluido mostrará la notificación.

## 9. Cómo se cumple cada requisito

| Requisito                         | Implementación                                                                                    |
|-----------------------------------|---------------------------------------------------------------------------------------------------|
| Solo el admin inicia sesión       | `LoginActivity` + `AuthRepository`; el resto entra directo a `MainActivity`.                      |
| Credenciales en código o pantalla | `Config.kt` (código) + diálogo de Configuración (DataStore).                                      |
| Carga de JPG/PNG/PDF              | `UploadActivity` con `OpenDocument` filtrando esos MIME; sube a Storage.                          |
| Visualización automática          | Listener en tiempo real de Firestore en `MainViewModel`; abre la circular más reciente al entrar. |
| Repetición cada 4 h por 2 días    | `WorkScheduler` (periódico 4 h) + `fechaExpiracion` = publicación + 2 días.                       |
| Reportes y estadísticas           | `ReportsActivity`: total, visualizaciones, activas/expiradas, fecha de envío y estado.            |
| Firestore + Storage               | `CircularRepository`.                                                                             |
| WorkManager + notificaciones      | `CircularDisplayWorker` + `NotificationHelper`.                                                   |

|                         |                |
|-------------------------|----------------|
| Compatible Android 8.0+ | `minSdk = 26`. |
|-------------------------|----------------|

## 10. Solución de problemas

- **Gradle Sync falla:** revisa conexión a internet y que tengas el SDK API 34. **File** → **Invalidate Caches / Restart**.
- **La app no muestra circulares:** confirma que reemplazaste `google-services.json`, que Firestore y Storage están habilitados y que el dispositivo tiene internet.
- **No llegan notificaciones (Android 13+):** acepta el permiso de notificaciones que pide la app al abrir.
- **WorkManager no se ejecuta exacto a las 4 h:** es normal; el sistema agrupa tareas para ahorrar batería (mínimo garantizado, no exacto). El periodo de 2 días sí se respeta por fecha de expiración.
- **PDF no se ve:** el visor usa Google Docs y requiere internet; verifica la URL del archivo en Storage.